TrainCache 实施步骤详解（白话版）

> 这份文档把《04-实施指南-P1》第 14 节那张 14 步的表格逐条展开，讲清楚每一步在做什么、为什么要做它、做到什么程度算过关。原文档里的设计细节（那些 § 编号）全部保留对应关系，随时可以翻回去看完整设计。写作原则只有一条：尽量说人话，每个术语第一次出现时顺手解释。

先弄清楚：两种章节，一条主线

原指南里有两类章节，分工不同。**§5 到 §10 按”零件”组织**，讲的是每个部件长什么样、内部怎么实现；**§14 那张表按”时间”组织**，讲的是零件按什么顺序装、装到哪一步算合格。

动手干活时，你以 §14 的步骤顺序为主线。每接到一步，先翻到对应的设计章节把细节读透，再开工。每一步都自带验收标准，验收不过就不往下走。这套流程跟游戏闯关差不多：一关一个 Boss，打完拿装备，装备还能帮后面几关省力。

名词小词典

下面这些词会在后文反复出现，先集中解释一遍，正文里就不啰嗦了。

名词	大白话解释
Kubernetes（K8s）	管理一大堆容器的系统。容器可以理解成轻量的”虚拟机”，一个容器跑一个程序。
Pod	K8s 里最小的部署单位，里面装着一个或几个容器。
kind	在自己电脑上用 Docker 模拟出来的迷你 K8s 集群，不用买服务器就能练手。
CRD	“自定义资源”。K8s 原生只认 Pod、Service 这类东西；CRD 相当于在 K8s 里登记一种新东西，之后就能用 yaml 文件创建和管理它。
Controller / reconcile	控制器是一个常驻程序，盯着某种资源，发现”现状”和”期望”不一致就去调整，每一次调整动作叫 reconcile（调谐）。
SchedulingGate	K8s 的调度暂停阀。给 Pod 挂上这扇门，调度器就先不给它分配节点，直到有人把门摘掉。
DaemonSet	一种部署方式：集群里每个节点都跑一份，适合装缓存、监控这类”每台机器都得有”的服务。
hostPath	把节点本机的某个目录直接挂给容器用，容器读写它就等于读写本机磁盘。
MinIO	开源的对象存储服务，相当于自己搭一个网盘。本项目用它扮演”又慢又远的云端数据源”。

名词	大白话解释
JuiceFS	把对象存储挂载成本地文件夹的分布式文件系统，是本项目的对照组之一。
Fluid	在 K8s 上管理数据缓存和预热的开源项目，本项目的头号对标对手。
DataLoad / Dataset	Fluid 里的两种 CRD：Dataset 登记一份数据集，DataLoad 登记一次预热动作。
tc / netem / tbf	Linux 自带的流量整形工具，能人为把网卡限速、加延迟，用来模拟”很慢的云存储”。
drop_caches	一条内核命令，把系统的文件缓存清空，让下一次读变成真正的”冷读”。
webdataset	把海量训练样本打包成一串大 tar 分片的数据格式，2026 年大模型训练的主流形态。
PyTorch DataLoader	训练程序里负责喂数据的组件。数据跟不上时，训练循环就会”饿着”等。
etcd	K8s 存所有元数据的小型数据库，单个对象最大 1.5MB，所以大数据清单塞不进去。
gRPC	服务之间点对点调用的通信框架，比裸 HTTP 更省事、更快。
/metrics	服务暴露监控数据的 HTTP 地址，Prometheus 会定时来抓取。
inotify / fsnotify	Linux 的文件事件监听机制：目录里有文件被打开、创建、修改，进程能立刻收到通知。
哈希 / 摘要（xxh3、SHA-256）	给一段数据算”指纹”的函数，内容变一点指纹就完全不同。xxh3 算得飞快，SHA-256 更正式但慢。
CAS（内容寻址存储）	按内容的指纹来命名和存放文件：内容相同的块只存一份，天然去重。
refcount（引用计数）	记录一块数据被几个地方引用着，计数归零了才允许清理。
LRU / LFU / W-TinyLFU	缓存满了之后”淘汰谁”的三种策略：LRU 淘汰最久没用的，LFU 淘汰用得最少的，W-TinyLFU 是带频率统计的高级版本。
singleflight	Go 的一个惯用法：同一时刻来了一堆相同的请求，只放一个去后端，大家共享结果。
AIMD	一种限流节奏：平时慢慢放宽，一出问题立刻减半，再慢慢恢复。
令牌桶	限流器：以固定速率往桶里放令牌，请求先拿令牌再出去，没令牌就等着。

名词	大白话解释
multipart 上传	把大文件切成许多片分开传，最后在服务端登记成一个完整文件。
一致性哈希环	决定”每块数据归哪个节点管”的办法；节点增减时只需要搬一小部分数据。
混沌测试	故意制造故障（杀进程、断网络、断数据源），看系统会不会像设计的那样自愈。
makespan	一个作业从提交到跑完的总时长，本项目最看重的指标。
dataloader stall	训练循环里”等数据”的时间占比。数据供给跟不上，CPU/GPU 就在干等。
fanout	预热时往几个节点同时复制数据。复制 3 份，字节成本就是 3 倍。

步骤 0：现勘——先侦察，再动工（已完成）

这一步的目标：动工之前先摸清两件事。第一，实验装置能不能稳定地造出”冷读慢、热读快”的可信差异；第二，对手 Fluid 的真实能力边界到底在哪。这两件事没确认之前写任何控制器代码，都属于闭眼开工。

具体做了什么：

1. **探针 A（装置可信吗）**：用 tc 把网络限速到 100Mbps、加 20ms 延迟，然后对比冷读（清空一切缓存后去 MinIO 拉数据）和热读（数据已在本地）的速度。两组各跑 7 次以上，中位延迟要差 5 倍以上，置信区间互不重叠，组内波动不超过 15%。
2. **探针 B（指标能测出来吗）**：写一个最小的 CPU 版 PyTorch 训练循环，读真实的 JPEG tar 分片，记录每一步里”等数据”的时间占比。冷态要稳定高于 40%，热态要低于 15%，而且增加 worker 数量后冷态等待要单调下降——噪声造不出这种单调性，所以它能证明测到的是真饥饿。
3. **部署对手并取证**：在 kind 集群上装好 MinIO、JuiceFS 社区版和开源 Fluid，把官方 demo 完整跑一遍，把 Fluid 实际注入的 Pod 配置、DataLoad 与调度之间的时序关系全部存档。
4. **产出现勘报告**：整理一张能力边界表，每个判断都附版本号和证据。

时间盒只有 3 天，红灯就去修装置，绝不带病开工。

> **你的进度**：这一步已经完成。确认双闸门（探针绿灯 + 缺口有证据）都过了之后，从步骤 1 继续往下走。

步骤 1：搭实验装置——你的”实验室”

设计细节见：§10.1、§10.2

为什么第一步是它：后面每做完一个零件，你都要立刻看到效果，而”看到效果”全靠这套装置。它必须稳定、可重复、一键启动，否则后面所有实验数字都不可信。

具体做什么：

1. **限速脚本**：把探针 A 时手工敲的 tc 命令整理成脚本。想快就快、想慢就慢，速度和延迟作为参数可调（比如 100Mbps/20ms 这一档），后面扫点实验全靠换档位。
2. **冷态规程**：写一个标准化脚本，每组实验开始前先执行 `sync; echo 3 > drop_caches` 清空系统缓存、重启相关进程，保证每一次”冷读”都一样冷。
3. **三轨数据集生成器**，三条生产线分别生产三种数据：
   - **A 轨**：生成 100 万个 4KB 小文件，模拟海量小文件场景，专门测元数据压力。这条轨只在 io_uring 支线里用。
   - **B 轨（主力）**：写脚本批量生成随机像素图片，做**真实的 JPEG 压缩编码**，再按 webdataset 规范打成百 MB 的大 tar 分片。解码开销是真实的，数据本身可以随手生成，不用下载 ImageNet。生成时固定随机种子，保证一键能再生成一模一样的数据。
   - **C 轨**：多 Job 倾斜数据（一份热数据加一份全量扫描数据），到步骤 8 之前备好就行。

过关标准：敲一条 `make bench-up`，整套装置（集群、存储、数据）自动拉起；限速档位可扫点；两次冷读的结果能对上。

步骤 2：写缓存核心库——项目的发动机零件

设计细节见：§5.1、§5.2

这一步的目标：写一个 Go 库，负责三件事——记录”有哪些数据块”、按内容把数据块落盘、按需下载它们。它此时没有任何服务形态，就是一组库代码加单元测试。

具体做什么：

1. **路径索引**：维护一份大清单，记录”某个路径下的第几块数据”存在哪里。一个数据集可能有几十万个块，清单本体太大放不进 etcd（上限 1.5MB），所以清单放在对象存储上，切成若干个索引分片管理，每个分片不超过 10 万条。数据集更新时靠 ETag 判断哪些分片没变，直接跳过，避免全量扫描。
2. **CAS 存储层**：数据块下载时会流过内存，你趁这个机会顺手算一个 xxh3 指纹，然后按指纹命名文件存盘（`blocks/ab/<指纹>.blk`）。这个设计一箭三雕：
   - 读回来时可以顺手校验数据有没有损坏（块头用 64 字节记录大小和摘要，不需要额外的校验文件）；
   - 不同路径下内容相同的块自动只存一份，比如多个分片的公共头部只占一份空间；
   - 每个块记一个引用计数，只有没人用的块才允许被清理。
3. **限并发抓取**：下载块时按”在途字节数”控制并发（默认上限 512MiB），而不是按 goroutine 数量。原因很实际：小文件会把 goroutine 数撑爆，大文件会把内存撑爆，字节数是两种情况统一的度量。同时限制对单个主机的连接数（默认 64）。
4. **写盘的原子性**：先写临时文件，写完再 `rename()` 改名。这样别的进程永远看不到写了一半的块。还有个反直觉的取舍：缓存块不做逐块 fsync——数据丢了重新下载就好，掉电丢块的代价就是重拉，这个取舍是正确的。

过关标准：单测全部通过，重点覆盖四种场景——多个 goroutine 同时写同一个块、写一半进程崩溃、相同内容去重、校验失败后重新下载。

步骤 3：Agent 服务化——给核心库装上对讲机

设计细节见：§5.3、§5.4

这一步的目标：把步骤 2 的库包成一个常驻服务（Agent），部署到每个节点上。别人能远程指挥它预热、查询它的进度，它也能主动汇报自己的状态。

具体做什么：

1. **gRPC 接口**：对外提供两个主要接口——“接收预热任务”和”上报块级进度”。控制器以后就靠这两个接口跟所有 Agent 说话。
2. **/metrics 监控**：暴露 Prometheus 格式的指标，指标上带 dataset 标签。注意控制标签的基数，别让标签组合数爆炸把监控系统撑垮。
3. **启动重扫描**：Agent 重启之后扫描缓存目录，把 CAS 索引重新建起来；对块头抽样校验 1%，全量校验放后台低优先级慢慢做。这是步骤 12 混沌测试能通过的地基。
4. **inotify 监听**：监听缓存目录里的文件打开事件（同一个进程反复打开同一个文件要合并），用来统计”哪些数据被读了”。要诚实：这套命中率是旁路统计的口径，Agent 不在读路径上，记不了精确账——报告里要写明这一点。

过关标准：手动调 gRPC 接口能下发预热、能查进度；`kill -9` 掉 Agent 再启动，索引自动恢复（这个恢复过程要有测试覆盖）。

步骤 4：数据面——让训练程序读得到预热好的数据

设计细节见：§3 的数据面约定

这一步的目标：把 Agent 铺满整个集群，并把训练程序读数据的路径约定下来。本项目的核心设计决策在这里：**Agent 不参与读路径**。

具体做什么：

1. **写 DaemonSet 部署文件**：Agent 以 DaemonSet 方式铺满所有节点，每个节点一个。
2. **约定读路径**：Agent 把数据块落在节点的固定目录 `/var/cache/traincache/<数据集>/blocks/`；训练 Pod 用 hostPath **只读**挂载这个目录。训练程序读数据直接走内核自带的 page cache，读路径上没有任何自研组件，所以读侧零额外延迟。代价是”谁读了什么”只能靠 inotify 旁路统计。
3. **写预热命令行工具（CLI）**：手动指定”把哪个数据集的哪些块预热到哪个节点”，用于调试和手动实验。
4. **验证整条链路**：手动预热完成后，起一个普通 Pod 挂载缓存目录，把数据全量读一遍。

过关标准：手动预热之后，普通 Pod 能完整读到全部数据。

步骤 5：Controller + 真实负载——第一次端到端串起来

设计细节见：§5.1、§6.1、§10.1

这一步的目标：把前面所有零件第一次串成完整链路，项目从”一堆零件”变成”一个系统”。这是端到端的 MVP。

具体做什么：

1. **定义两个 CRD 并写控制器**：Dataset 登记数据集，预热任务登记一次预热。控制器的日常工作是一个循环：盯着训练 Job 的声明 → 挑出候选节点 → 给这些节点上的 Agent 下发”这个 Job 要用哪些块”的预热任务 → 收 Agent 的进度汇报 → 把就绪度回填到 CRD 的 status 里。训练 Job 通过 label 或 annotation 声明自己要消费哪些分片（大 Job 往往只用数据集的一个子集）。
2. **接入真实 dataloader**：用 PyTorch 的 DataLoader 读 B 轨的真实 tar 分片，跑一个不带模型（或挂极小矩阵乘）的 CPU 训练循环，打点记录 `wait_time/step_time`。设计文档特别强调：**真实负载在这里就要接入，不许推到最后再做**。
3. **保证幂等**：控制器重启、事件重放都不能产生重复任务。informer 的事件可能重复送达，你的 reconcile 逻辑必须做到”执行两次和执行一次结果一样”。

过关标准：从”提交一个声明了数据子集的 Job”到”数据在候选节点预热完成、训练 Pod 能读到数据”，整条链路无人工干预跑通。

步骤 6：基准框架，跟 Fluid 上秤——第一个简历成果

设计细节见：§10.3

这一步的目标：把五组对照实验自动化，让”跟对手同台跑分”变成一条命令的事。学项目最大的原罪是不敢上秤，这一步就是给项目装秤。

五组对照（同一份负载轨迹、同一套冷态规程、同一台机器）：

组	条件	含义
1	直读 MinIO	完全不缓存的零缓存基线
2	JuiceFS 冷读	对手首次访问，缓存是空的
3	JuiceFS 预热后热读	对手预热完成后的效果
4	Fluid 完整流程	按官方姿势部署 Dataset + DataLoad + 亲和调度，最强工业基线
5	TrainCache 自己	required 门控、preferred 门控、超时降级三种策略分别跑

具体做什么：

1. 写实验编排脚本，保证五组实验吃同一份固定随机种子的请求轨迹文件；
2. **每组至少跑 7 次取中位数，并报告 bootstrap 95% 置信区间**——只跑 3 次取中位数的结果不许进报告，这是硬纪律；
3. 环境全部锁定入库（镜像 digest、软件版本、tc 参数、随机种子），做到 `make bench COND=xxx` 一键复现任何一组。

过关标准：至少先打通第 1、2、3、5 组的自动化对比，能一键产出对照报告和图。**这一步完成时，你手里就有了第一个能写上简历的成果：包含真实对手的对照报告。**第 4 组（Fluid 全家桶）可以稍后补上，但最终必须补齐。

步骤 7：核心 1 预热-调度闭环——项目灵魂

设计细节见：§6

这一步的目标：实现整个项目的灵魂——“预热-调度闭环”。完整回路是：Job 提交 → 控制器往候选节点预热它要的数据 → 数据就绪度够了才放行调度 → 超时就降级放行。同时产出整个项目最重要的一张图。

具体做什么：

1. **SchedulingGate 门控**：Job 创建时，控制器先给它挂上”暂缓调度”的门，数据就绪后摘掉。挂门摘门必须幂等，还要用 finalizer 防止删除时留下孤儿对象。要求 K8s 版本不低于 1.26。
2. **就绪度按”子集”算**：训练 Job 声明自己只消费哪些分片，就绪度只看这些分片在候选节点上的覆盖比例（默认 90% 放行），不看整个数据集。这是和 Fluid”节点上有缓存 runtime 就算数”的本质区别。
3. **Score 打分插件**：调度器选节点时，按”每个节点持有这个 Job 要用的块的比例”打分，节点空闲磁盘和在途预热负载做次要排序。**插件内部禁止发网络请求**，用 informer watch 聚合状态加本地缓存（1 秒 TTL）——在调度器插件里发网络请求把调度器打挂，是这个领域的经典大坑。
4. **超时降级**：预热总超时默认 5 分钟（注解可覆盖）。超时之后去掉约束正常调度，写入 Degraded 状态、记录事件和指标。**Job 永远不会因为预热问题而死锁在 Pending**，这句话是你的设计底线。门控分两档：required（硬等，等不到就超时降级）和 preferred（软门控，超时立即放行）。
5. **预热限速**：预热速率受预算约束（默认不超过节点空闲带宽的一定比例），防止预热点火时挤压正在跑的 Job。
6. **旗舰实验——盈亏平衡曲面**：扫三个变量（源站带宽、数据集/缓存容量比、fanout），画出”闭环门控比自由调度快还是慢”的正负区间。**假设必须在实验前写死（预注册）**，结果是什么就报什么，禁止看到结果再调参。这张图哪怕大半区域是负的也算成功——它回答了”这东西什么时候不该用”，而这个问题恰恰是面试官最想听的工程判断。
7. **配套次旗舰图**：fanout 取 1/2/3 时，makespan 收益与重复预热字节成本的 Pareto 散点图。

过关标准：门控、打分、降级全链路工作；盈亏平衡曲面和 Pareto 图产出；插件内无网络调用。

步骤 8：核心 2 缓存淘汰对照——缓存不够大时谁留下

设计细节见：§7

这一步的目标：回答”缓存装不下全部数据时，LRU、LFU、W-TinyLFU 三种淘汰策略到底差多少”。这个差异只在特定场景出现，所以你要主动构造场景，同时坦然接受”没差异”的结论。

为什么必须构造场景：全量顺序训练加缓存足够大时，三种策略的命中率会趋同，比较出来必然是一条平线。策略差异只在”缓存欠配、访问倾斜、热数据被重复消费、多租户互相挤占”同时成立时才显现。

具体做什么：

1. **构造倾斜场景（C 轨）**：Job A 反复读 10% 的热分片（模拟多轮微调和重复 epoch），Job B 顺序扫全量数据（流寇式扫描），缓存只给数据集的 30%。
2. **三种策略同台**：记录 Job A 的命中率和等待占比，以及 Job B 把热数据挤出去的次数。W-TinyLFU 按规格实现：window 加两段 SLRU、count-min sketch 频率统计（4 行、宽度取缓存块数的 10 倍、3bit 饱和计数）、doorkeeper bloom 过滤器，准入规则是”新块的频率高于被淘汰块才换入”，定期做减半压缩。

3. **自适应预取**：inotify 观察到第 i 块被读，就预取后 8 块；预取命中率低于阈值就自动把 8 调小。
4. **参数敏感性扫点**：window 占比、减半周期等参数各扫一遍画图。

允许的阴性结论：如果顺序场景下三种策略没差异、倾斜场景下才有差异，就如实写”准入策略对单 Job 的顺序训练并不关键，真正关键的是预取调度与跨租户保护”。这是一个成熟的阴性发现，算不上实验失败。

过关标准：倾斜场景差异曲线 + 顺序场景阴性结论 + 参数敏感性图。

步骤 9：S3 合并回源网关——同一份数据只回源一次

设计细节见：§8.1

这一步的目标：写一个网关挡在 Agent 和 MinIO 中间。多个节点同时要同一个对象时，只放一个请求去后端，大家共享结果，替中心存储省带宽。

具体做什么：

1. **singleflight 合并**：按对象 key 合并并发请求，同一时刻只有一个请求真正打到 MinIO。
2. **流式扇出**：大对象用 `io.Pipe` 边拉边分发给所有等待者，禁止整块缓冲完再分发——那样内存会炸。
3. **AIMD 背压**：盯着 MinIO 的 P99 延迟，延迟升高或出现 5xx 就把令牌桶上限立刻减半，平稳期再线性恢复。并发上限本身就是”对中心存储的礼貌”。
4. **内存复用**：用 `sync.Pool` 复用块级 buffer，开 `GODEBUG=gctrace=1` 对比每字节分配数和 GC 停顿，作为 Go GC 调优的热身。

过关标准：压测大对象扇出，内存曲线不涨（要用压测数据证明）。

步骤 10：Checkpoint 异步 sidecar——存档不卡训练

设计细节见：§8.3

这一步的目标：训练程序存档（checkpoint）时写本地就返回，由一个旁路容器异步上传到对象存储；恢复时并发拉取，把加载和初始化重叠起来。

具体做什么：

1. **写路径**：trainer 把 checkpoint 写到共享的 emptyDir 目录，sidecar 通过 fsnotify 感知新文件，切成 16MB 定长分片、逐片算 xxh3、8 个并发做 multipart 上传，**最后写一个 manifest 对象作为发布点**——S3 上永远只能看到发布后的完整 checkpoint，传一半的状态对外不可见。校验以 manifest 里的逐片哈希为准（multipart 的 ETag 不是 md5，别用它校验）。
2. **恢复语义（这里 v3 修正过一个重要错误）**：训练恢复时，权重和 optimizer state（比如 Adam 的动量）**都完整之后才允许跑第一个 step**——缺了 optimizer state 继续训练会改变收敛轨迹，这是 v1 的语义错误。sidecar 真正的价值是”异步上传不阻塞训练循环，恢复时用 Range GET 并发拉取，加载与初始化重叠以缩短等待”，它省不下状态本身。
3. **指标**：checkpoint 写入对训练 step 的扰动、恢复总时长的拆解（拉取、加载、与初始化重叠的部分）。

过关标准：恢复时状态必须完整才能跑 step；恢复时长有完整拆解。

步骤 11：P2P 缓存组——节点之间互相借数据

设计细节见：§ 8.2

这一步的目标：JuiceFS 的”分布式缓存组”是企业版才有的能力。你在社区版（只有单机缓存）的基线上，用 Go 把这套语义自己复现出来：节点间互相要数据，实在没有才回源。

具体做什么：

1. **节点发现复用 CRD status**：每个节点上报自己持有哪些块（位图摘要），不需要额外搭发现服务。
2. **一致性哈希环定”老家”**：每块数据在环上有一个”老家节点”；拉取顺序是先找 peer，最多同时向 2–3 个 peer 多源拉取，peer 都没有才回源；peer 请求失败立刻穿透回源——这与企业版文档描述的故障语义一致，peer 故障并不致命。
3. **带宽预算**：每个节点一个令牌桶，peer 流量和回源流量共用，回源保底占 20%，防止 P2P 流量把训练流量挤掉。
4. **Transport 接口抽象**：传输层定义成接口，默认实现是 gRPC stream；RDMA 作为实验臂插在接口后面（见支线任务），永远不进主路径。

过关标准：源站出口字节数下降；回源保底生效。

步骤 12：混沌套件——主动搞破坏，验证扛得住

设计细节见：§ 8.4

这一步的目标：故意制造四种故障，逐一验证系统行为符合设计预期。前面每一步埋的自愈能力（重扫描、背压、校验重拉、超时降级），在这里集中验收。

注入的故障	手段	期望的表现
Agent 进程崩溃	kill -9 再等重启	重扫描后恢复索引，没有数据错误
网络劣化	tc/netem（只在特权测试容器里用）	背压生效，P99 恶化但有界
数据损坏	测试钩子篡改块字节	校验失败 → 自动重拉 → 有指标记录
数据源宕机	把 MinIO 缩容到 0	门控超时后降级，Job 不卡死

过关标准：四件套全部通过，恢复时间指标齐备。

步骤 13：真值校准——把笔记本数字换成真机器数字

设计细节见：§ 10.4

这一步的目标：单机上的冷热差是 tc 限速人为造出来的，绝对带宽数字要用真机器校准一次。注意 P1 全程不需要 GPU，租 CPU 云主机就够，省钱。

具体做什么：

1. 租两台带独立数据盘的 CPU 云主机（数据盘和系统盘分开）；
2. 挂 tc 校准网络路径，把五组对照实验全量复跑一遍；

3. 用真存储数字替换单机数字，**两个版本的数字都保留**，并在报告里说明差异来源。

过关标准：报告里同时有单机版和真机版数字，差异解释清楚。

步骤 14：上游回馈——把数据送给 Fluid 社区

设计细节见：§1 三重定位、§6 的数据

这一步的目标：拿着步骤 7 的盈亏数据和现勘证据，向 Fluid 项目提 issue，争取发展成 PR。第三方社区的背书是简历上信用最高的东西，这一步做成了，前面所有测量工作就有了双重回报。

具体做什么：

1. 把测量数据整理成对维护者有说服力的 issue（现象、数据、复现方法、建议）；
2. 提交 issue → 跟维护者讨论 → 按维护者的认可程度推进 PR（调度/DataLoad 方向）。

过关标准：至少 1 个带测量数据的 issue；PR 不强求合并，有实质进展即可。

支线任务（不在关键路径上，按兴趣推进）

- **RDMA 实验臂**（挂在步骤 11 之后）：在笔记本上用 Soft-RoCE 跑真正的 verbs 代码，只验证功能正确性，**性能数字一律不进报告**（Soft-RoCE 常常比 TCP 还慢）；真值按小时租阿里云 eRDMA 校准一次，几十块钱的事。别把 libibverbs 用 cgo 链进 Go 热进程，用一个独立的 C++ 传输组件加薄薄的 C ABI。
- **io_uring**（步骤 8 前后）：借 A 轨小文件数据独立推进。诚实的预期：4MiB 大块抓取受网络限速，省 syscall 对吞吐贡献接近零；真正的收益在小文件高 IOPS 场景。O_DIRECT 的三重对齐、注册缓冲、fio 校验都照做。
- **GPU 侧技术（DALI / GDS / CUDA Stream）**：只写半页调研笔记，完全不动手。面试口径固定为”我没有动手做，但我知道它在流水线上和我的系统怎么接力”。

排期锚点与砍项顺序

- **2027 年 2 月实习投递包的最低构成**：完成步骤 0–7，外加至少 1 个带数据的 Fluid issue。也就是基座加核心 1（五组对照至少跑完三组，加上盈亏平衡旗舰图）。
- **2027 年 8 月秋招包**：完成步骤 0–11、13、14（issue 有维护者正式回应，或 PR 有实质进展）。
- **时间不够时从后往前砍**：RDMA 臂 → FUSE → 混沌里的位翻转用例 → P2P 的高级部分（多源调度）→ W-TinyLFU 的完整版（简化版已足够进简历）。**核心 1、B 轨真实负载、Fluid 对照这三样任何时候都不砍。**